

嵌入式操作系统

Embedded Operating System

信息与软件工程学院

桑楠

sn@uestc.edu.cn

2026年3月

第四部分

编写内核

—— 内存管理

核心内容

- 嵌入式内核的基本构成
- 任务管理
- 事件处理机制
- 内存管理机制
- 任务交互机制

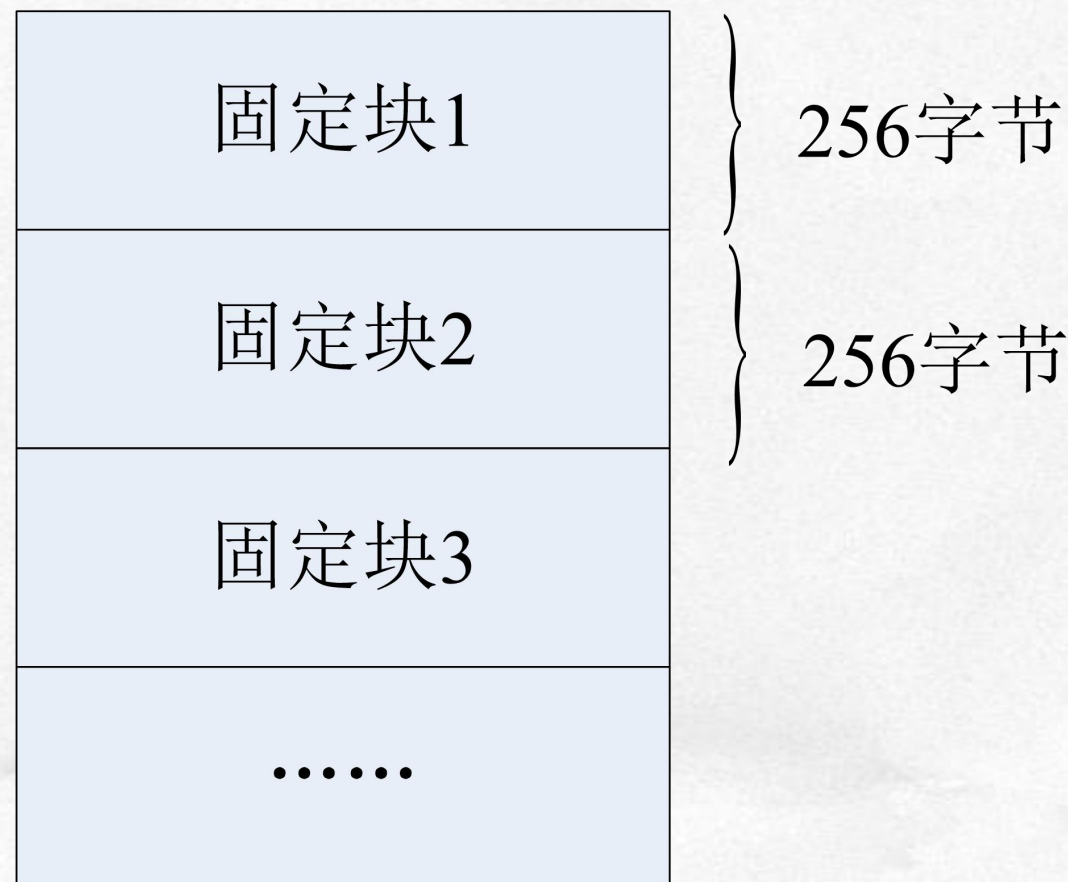
内存管理机制

主流内存管理机制

- **固定大小储存管理：**

内存由一段**连续的存储**构成，这段存储被分为多个大小一样的**固定块**。

- **存在问题：**无法解决内存碎片问题。



主流内存管理机制 (2)

- **可变大小储存管理**：内存由一段**连续的存储**构成，这段存储被分为多个大小不一样的存储块。
- **任意大小分配**：**需要多少分多少**——可用内存中划分出所需一块，剩余以后再用；**管理复杂**——剩余可用项，链表且每项大小不一。
- **固定大小可变分配**：可分配内存块大小相对固定——**如2的幂次**。每块的大小可变。

主流内存管理机制 (3)

- 三种机制的比较

- 固定大小 没有外碎片，分配回收速度快，分配时间确定性好；若使用不当，碎片很大
- 任意大小 很少内碎片；外碎片比较严重，分配回收速度慢，分配时间确定性差。
- 固定大小可变 前两种方式的结合体，各种性能处于中间位置

RTOS内存管理基本思想

- **存储管理**：对RTOS和应用所用内存资源的管理方法，
内存分配和内存回收
- 实时内核所采用的内存管理方式与应用领域和硬件环境密切相关
- **强实时应用领域**：内存管理方法就比较简单，甚至不提供内存管理功能
- **复杂实时应用领域**：可靠性要求比较高，且系统比较复杂的应用在内存管理上就相对复杂些，可能需要实现对操作系统或是任务的保护

需要关注的因素

- **快速性：快速而确定的内存管理；** 内存分配或回收算法 $O(1)$
 - **不使用内存管理：** 最快速和最确定的内存管理方式，适用于那些小型的嵌入式系统，系统中的任务比较少，且数量固定
 - **基本的内存管理方法：** RTOS提供内存分配与释放的系统调用。
- **高效性：内存应该各尽其用**
 - **内存分配尽可能减少浪费：** 不可能为满足所有用户需求而将内存配置得无限大 —— **避免通用**

矛盾！！

需要关注的因素 (2)

- **不使用虚拟存储技术**
- **虚拟存储原理** 每个任务从内存中获得一定数量的页面，并且，当前不访问的页面将被置换出去，为需要页面的其他任务腾出空间
- **虚拟存储置换** 一种具有不确定性的操作 —— 无法确定内存使用的时间
- **结论** RTOS中一般不使用虚拟存储技术，以避免页面置换所带来的开销

内存保护

- **防止地址越界**：每个应用程序都有自己独立的地址空间，当应用程序要访问某个内存单元时，由硬件检查该地址是否在限定的地址空间之内，只有在限定地址空间之内当的内存单元访问才是合法的，否则需要进行地址越界处理。
- **防止操作越权**：对于允许多个应用程序共享的存储区域，每个应用程序都有自己的访问权限，如果一个应用程序对共享区域的访问违反了权限规定，则进行操作越权处理。

通用的内存分配

- **内存空间划分**：代码区，静态数据区和**动态数据区**
 - **堆 (heap)**：一种链式结构；动态分配
 - **栈 (stack)**：一种线性结构；每个任务有私有“栈”
- **静态分配**：发生在编译和连接时，**全局变量和静态变量**；开始运行就分配，**运行结束才释放**
- **动态分配**：发生在程序调入和执行时，**局部变量和返回值等**；需要时分配，**使用完就释放**
 - **栈**：函数的参数和返回值、局部变量、...
 - **堆**：malloc和free分配/释放的内存，如指针。
 - **内存泄漏**：malloc分配，但缺少free释放

显式释放

通用的内存分配 (2)

// 栈内存分配

```
char* AllocStrFromStack()
{
    char pstr[100];
    return pstr;
}
```

退出释放

// 堆内存分配

```
char* AllocStrFromHeap(int len)
{
    char *pstr;

    if ( len <= 0 ) return NULL;
    return ( char* ) malloc( len );
}
```

未释放

通用的内存分配 (3)

// VC程序

```
int main() {  
    int ** matrix;  
  
    for (;;)   
        matrix = (int **)new int[100][100];  
    return 0;  
}
```

死循环，不断分配，不回收：
内存溢出

```
// 得到的字符串在使用完后  
// 必须使用delete删除Char *  
get_string() {  
    char * ret = new char[100];  
  
    strcpy(ret, "Test String!");  
    return ret;  
}
```

已经说明使用方法：
但经常失效！！

通用的内存分配 (4)

- **malloc和free的使用规则** —— 避免内存泄漏等
 - **配对使用**：一个malloc，必须有一个或多个free —— **确保每条路径释放**
 - **尽可能在同一函数中使用**：避免malloc在函数中，而free在函数外 —— **无法准确判定是否释放**
 - **初始化**：malloc分配的内存一定要初始化 —— **否则其值可能是以前的**
 - **释放**：free后的指针一定要设置为NULL —— **否则指针内容未改变，还可以取值使用，即“野指针”**

通用的内存分配 (5)

```
#include <stdlib.h>
```

```
int main() {
```

```
    char *str;
```

```
    if ((str = malloc(15)) == NULL) {
```

```
        printf("Not enough memory to allocate buffer\n");
```

```
        exit(1);
```

```
    }
```

```
    // 内部处理
```

```
    strcpy(str, "hello world");    // 初始化
```

```
    printf("String is %s\n", str);
```

```
    free(str);
```

```
    str = NULL;
```

```
}
```

局部指针

动态分配

释放：指针
依然可用

指针不可用

RTOS的内存分配

- **不考虑静态分配问题、栈空间问题**
- **动态分配：**理论上，malloc()和free()
- **存在问题：**
 - 创建任务、队列、互斥量、软件定时器、信号量或事件组时，RTOS内核会为它们分配RAM
 - 会占用更多宝贵的代码空间
 - 没有任务（**线程**）保护：**不保证资源分配使用回收正确**
 - 不具有确定性：**每次调用执行的时间可能不同**
- **结论：**并不总是可以使用的
- **解决方案：**提供个性化的内存分配方案 —— **针对不同应用**

FreeRTOS内存分配模式之一

- **原则：**一旦分配之后，不允许释放分配的内存
- **应用对象：**硬实时嵌入式应用，大多数只是在系统启动时创建所有任务、队列、信号量等，直到程序结束都会一直使用它们，永远不需要删除
- **特点：**
 - 用于从不会删除任务、队列、信号量、互斥量等的应用程序
 - 执行时间确定，不会产生内存碎片
 - 实现和分配过程非常简单：从一个静态数组中获得

无动态分配

RTOS内存分配模式之二

- **原则：**使用一个**最佳匹配算法**，允许释放之前分配的内存块，但不合并相邻空闲块——**可能会造成内存碎片**
- **应用对象：****动态创建任务的小型实时系统**；不能用在分配和释放随机字节堆栈空间的应用程序
- **特点：**
 - 动态创建和删除任务，各任务的堆栈空间大小相同
 - 动态创建和删除队列，且各队列存储区域大小相同（项数不定）
 - 直接调用pvPortMalloc()和pvPortFree()函数，**不使用OS API**
 - 重复的分配和删除具有相同堆栈空间的任务、队列、信号量、互斥量等等——**使用同一存储空间**
 - 应用程序中的队列、任务、信号量、互斥量等等处在一个不可预料的顺序，有可能导致内存碎片问题
 - 不具有确定性，但比标准库中的malloc函数具有更高的效率

本模式的分配/
回收函数

μcOS内存分配算法DSA

- **DSA**: Dynamic Storage Algorithms
- **应用对象**: 没有碎片管理能力, 需要预知要分配的内存大小
- **算法思想**:
 - **整个内存按用户需求进行分区**: 多个大小不一的分区, 且每个分区中的空闲块数由用户自己指定
 - **管理内存**: 内存控制块; 每个分区一个
 - **管理结构**: 所有分区链接成一个表格
 - **请求内存**: 内存控制块→选择分区→选择空闲内存块。以后的操作针对该内存管理块
 - **释放内存**: 当前内存块空闲→指定在原分区控制块中
 - **用户的操作**: 需要知道每个控制块的内存大小, 以及哪些是个空闲的

μCOS内存分配算法DSA (2)

// 内存控制块的数据结构

```
typedef struct {  
    void * OSMemAddr;  
    void * OSMemFreeList;  
    INT32U OSMemBlkSize;  
    INT32U OSMemNBlks;  
    INT32U OSMemNFrees;  
} OS_M
```



RTOS内存分配模式之三

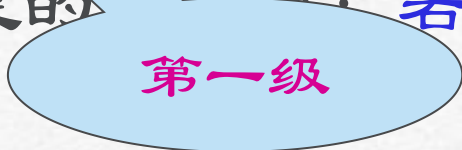
- **原则：** 使用malloc()和free()
- **方法：** 包装了标准库中的malloc()和free()函数，包装后的malloc()和free()函数具备任务（线程）保护
- **特点：**
 - 需要链接器设置一个堆栈，并且编译器库提供malloc()和free()函数
 - 不具有确定性
 - 可能明显的增大RTOS内核的代码大小

RTOS内存分配模式之四

- **原则：** 使用一个最佳匹配算法，将相邻的空闲内存块合并成一个更大的块（包含一个合并算法）
- **应用对象：** 特别适用于移植层代码，可以直接使用 `pvPortMalloc()` 和 `pvPortFree()` 函数来分配和释放内存
- **特点：**
 - 可用于重复分配、删除任务、队列、信号量、互斥量等等的应用程序
 - 可用于分配和释放随机字节内存的情况，不会如模式之二那样产生严重碎片
 - 不具有确定性，但是它比标准库中的 `malloc` 函数具有高得多的效率

μCOS: TLSF算法的原理

- **TLSF（二级间隔表动态内存分配算法）** **算法思想：**

- **堆内存的管理由三个链表完成：**地址链表、空闲内存块链表、使用内存块链表
- **请求内存块：**首先在空闲链表中寻找满足的内存块，若无： **若无：**再在堆中获取一块内存块
- **释放内存块：**将释放后的内存块顺序插入到空闲链表中
- **地址链表：**记录所有划分出的内存块，按地址顺序链接
- **使用链表：**记录正在使用的内存块，不排序
- **空闲链表：**记录空闲的内存块，按从小到大排序
- **专用任务：**负责相连空闲块的合并——当系统处于空闲状态时，系统进入该内存专用管理任务

μCOS: TLSF算法的原理 (2)

- **TLSF的内存控制块结构**

```
typedef struct mem_control_block {  
    unsigned int is_available;           //是否有效标记  
    unsigned int size;                   //分配的用户可用内存空间大小  
    struct mem_control_block * p_addr_next; //初始化的地址链表  
    struct mem_control_block * p_size_next; //初始的空闲链表，按大小排序  
    struct mem_control_block * p_used_next; //初始的使用链表，按使用过程排序  
    void * addr;                         //用户使用的内存块首地址  
};
```

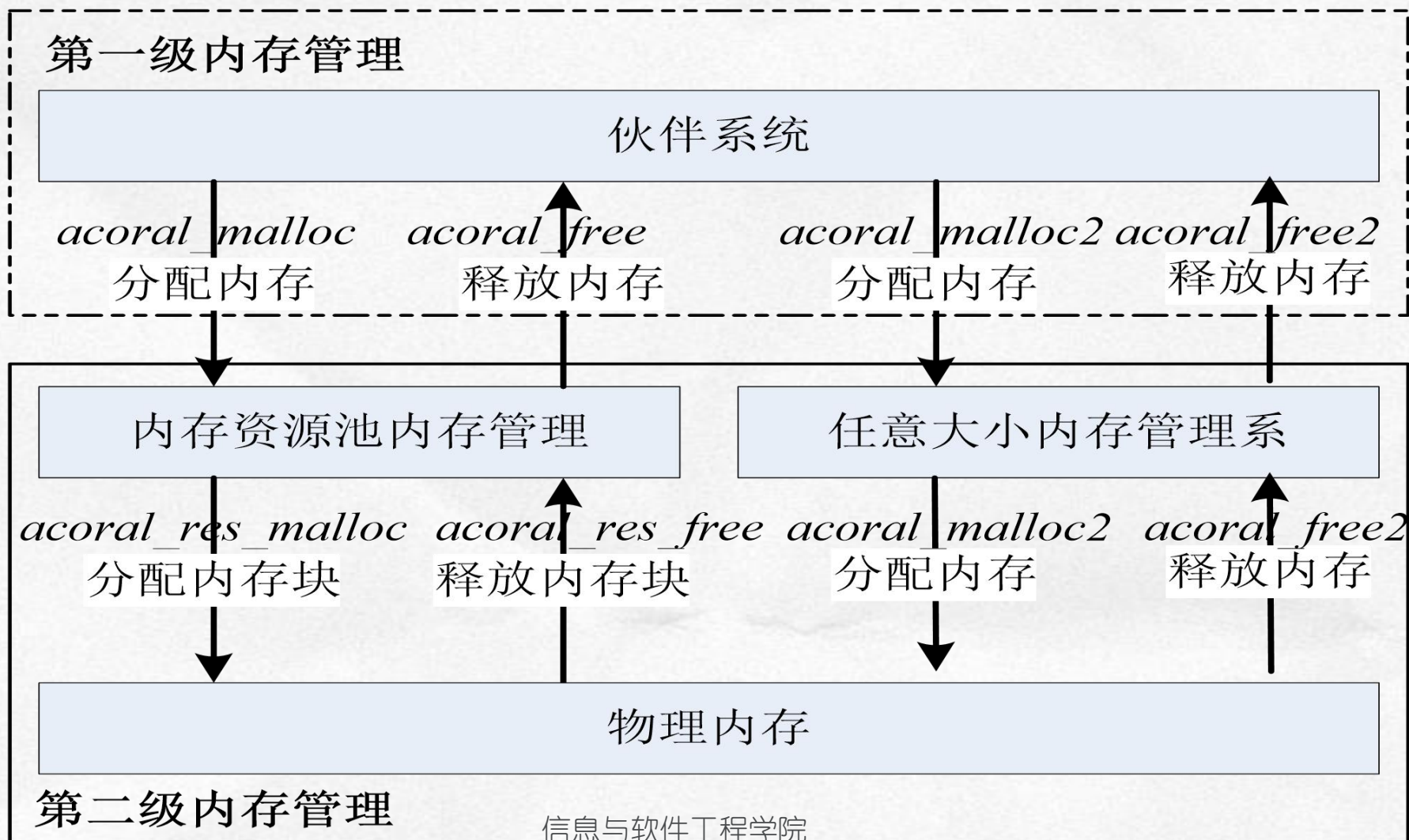


aCoral

内存管理机制

aCoral的内存管理模式

- 原则：两级管理模式



aCoral的内存管理模式 (2)

- **第一级：改进的伙伴系统**

- 划分要分配的内存块大小
- 每个内存块分配 2^N 大小内存
- 解决内存外碎片和内碎片问题

- **第二级：根据第一级确定的大小进行具体物理内存分配**

- 固定块和可变大小可选

- **使用方法：**

- 应用程序 直接使用第一级管理模式
- 内核 可直接使用第二级管理模式

伙伴关系管理算法

- 整个可用空闲内存由 2^m 个字节组成：假设内存地址从 $0 \sim (2^m - 1)$
- 在对系统中每个大小为 2^n ($0 \leq n \leq m$) 的内存块建立一个对应的可用块链表
- **初始化**：整个 2^m 个字节空间都是可用的
- 应用程序申请 2^K 个字节的内存空间
- 若存在 2^K 大小的空闲内存块，则直接使用
- 若没有 2^K 大小的内存块时，就把更大的可用内存块分成两部分
→ 最终得到两个大小为 2^K 的内存空间
- 当一块分成两块时，这两块就称为彼此的伙伴
- **合并**：某个时刻若这两个伙伴都空闲，可合并成一个大的空闲块

伙伴关系管理算法 (2)

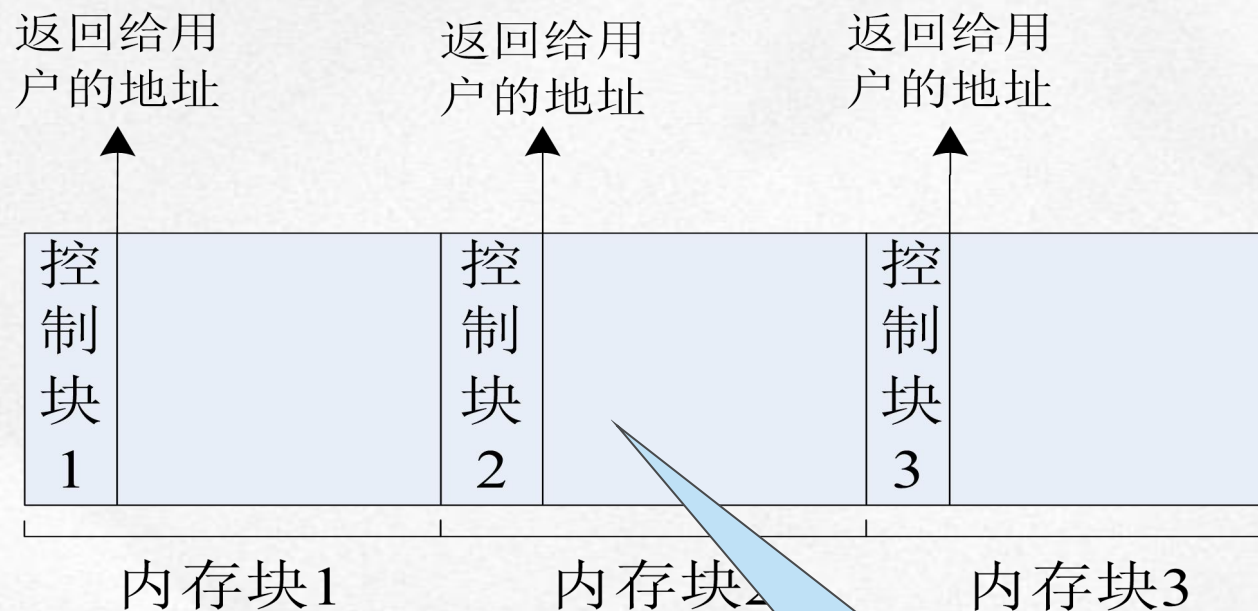
- **优点：**

- 算法简单，分配和回收速度快
- **合并简单**：一个大小为 2^K 字节的块释放后，只需要搜索 2^K 字节大小的块以判定是否需要合并——不需要搜索整个空闲块表

- **缺点：**

- 虽然合并简单，但合并所需时间无法确定：因链表只能顺序搜索， $O(n)$ → **是否能实现一种 $O(1)$ 算法？**
- 内存控制块的组织问题：在内存块开始部分预留出一段内存来作为内存控制块 → **预估空闲块大小有误**

伙伴关系管理算法 (3)



- 假定控制块大小为 N ，用户需要分配 2^k 内存块
- **实际所需大小**： $2^k - 2N + N = 2^k - N$
- **不清楚管理模式**：申请 2^k ，**实际**申请 2^{k+1}
- **清楚管理模式**：申请 $2^k - 2N \sim 2^k - N$

问题：如何解决？
用户编程控制？
改进算法？

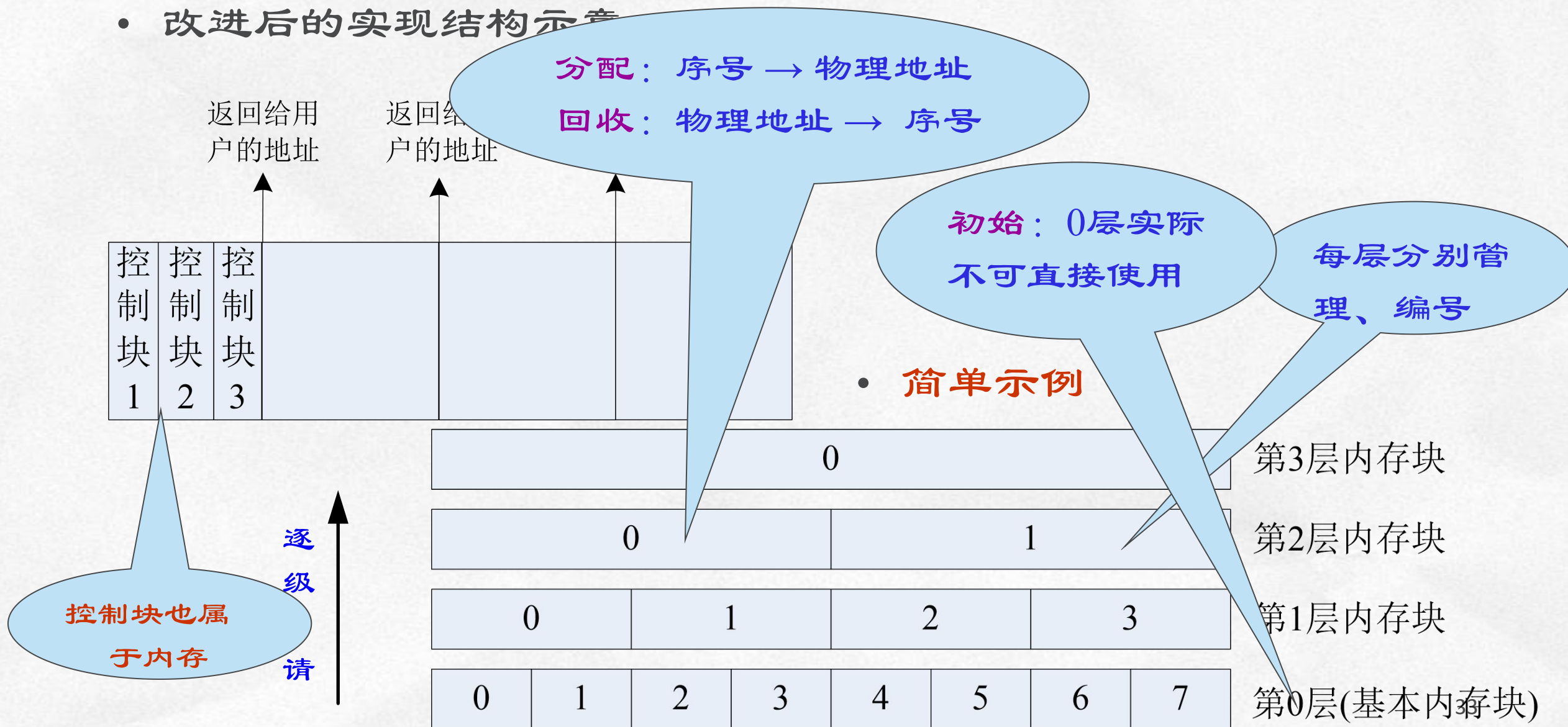
aCoral的第一级管理算法：伙伴关系改进

- **原则**：单独处理控制块，内存块大小实际可用
- **思想**：
 - 实际物理内存分为两部分：内存控制块所用、用户可用
 - 内存初始化函数buddy_init()逐个初始化这些管理结构
 - 可用内存被划分为众多基本块，每个基本块的大小为常量BLOCK_SIZE (默认1~7字节，系统可配置)
 - 所有基本块按0~N编号，并分为m层 (0~14) 管理
 - 0层大小为 $2^0 * \text{BLOCK_SIZE}$ ，1层为 $2^1 * \text{BLOCK_SIZE}$
 - 采用伙伴关系算法思想管理这些基本

$\log_2(2^i / \text{BLOCK_SIZE})$
求出属于第几层

aCoral的第一级管理算法：理解

改进后的实现结构示意图



aCoral的第一级管理算法：理解（2）

- 假定共有 2^n 个基本内存块（假定 $n=3$ ），分为 m 层($m=4$)
- 初始化： $2^n(8)$ 个内存块均空闲，注册在第 $m-1$ 层（3，最高层）为一个整体；以下各层不可用
- 当需要分配 2^i 字节大小的内存块时，用公式： $\log_2(2^i / \text{BLOCK_SIZE})$ 求出应该从哪一个逻辑层分配内存——假定 $i=0$ ，一个基本块。
- 0层不可用：逐级申请到3层（顶层）→ 划分为两个 2^2 内存块 → 选择一个，划分为两个 2^1 内存块 → 选择一个，划分为两个 2^0 内存块
- 最终选择一个 2^0 字节大小的内存块，分配！
- 回收：逆向；哪层分配，回收到哪层

aCoral的第一级管理算法：内存控制块

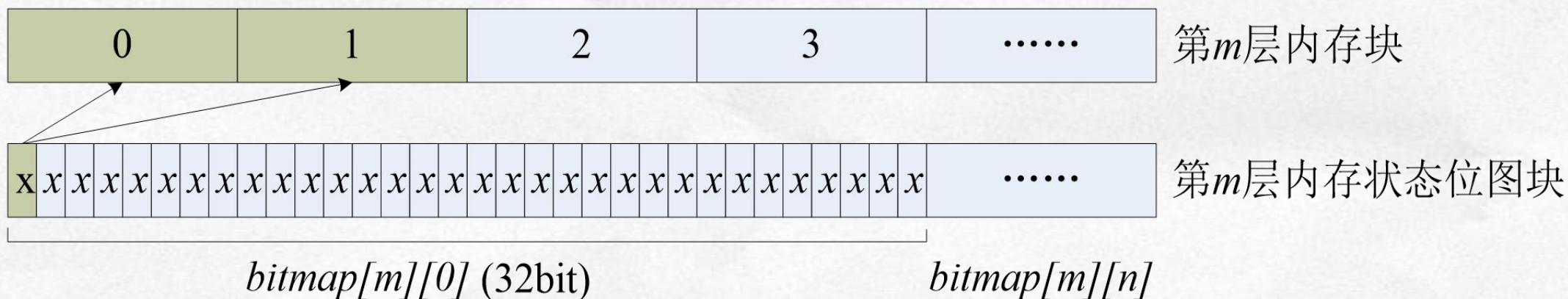
- 每层需要三个结构：内存状态位图块、空闲内存块链表数组、空闲内存块链表头
- 需要为每个基本内存块存储逻辑层信息，即属于哪一个逻辑层

```
typedef struct{  
    acoral_32 *free_list[LEVEL]; //各层空闲内存块链表  
    acoral_u32 *bitmap[LEVEL]; //各层内存状态位图块  
    acoral_32 free_cur[LEVEL]; //各层空闲内存块链表头  
    acoral_u32 num[LEVEL]; //各层内存块个数  
    acoral_8 level; //层数  
    acoral_u32 start_adr; //内存起始地址  
    acoral_u32 end_adr; //内存终止地址  
    acoral_u32 block_num; //基本内存块数：0层数  
    acoral_u32 free_num; //剩余基本内存块数  
    acoral_u32 block_size; //基本内存块大小  
    acoral_spinlock_t lock; //自旋锁  
} acoral_block_ctr_t;
```

aCoral的第一级管理算法：三种结构的作用

- 内存状态位图块bitmap：

- 一个二维数组 $\text{bitmap}[m][n]$ ： m 代表层数， n 代表某层的第 n 个位图块
- 每个值 $\text{bitmap}[m][n]$ ： 32位， 每位值0或1， 可表示64个内存块
- 每位： 0代表无空闲， 1代表有1个空闲伙伴
- 不存在两个伙伴均空闲： 否则归为上层



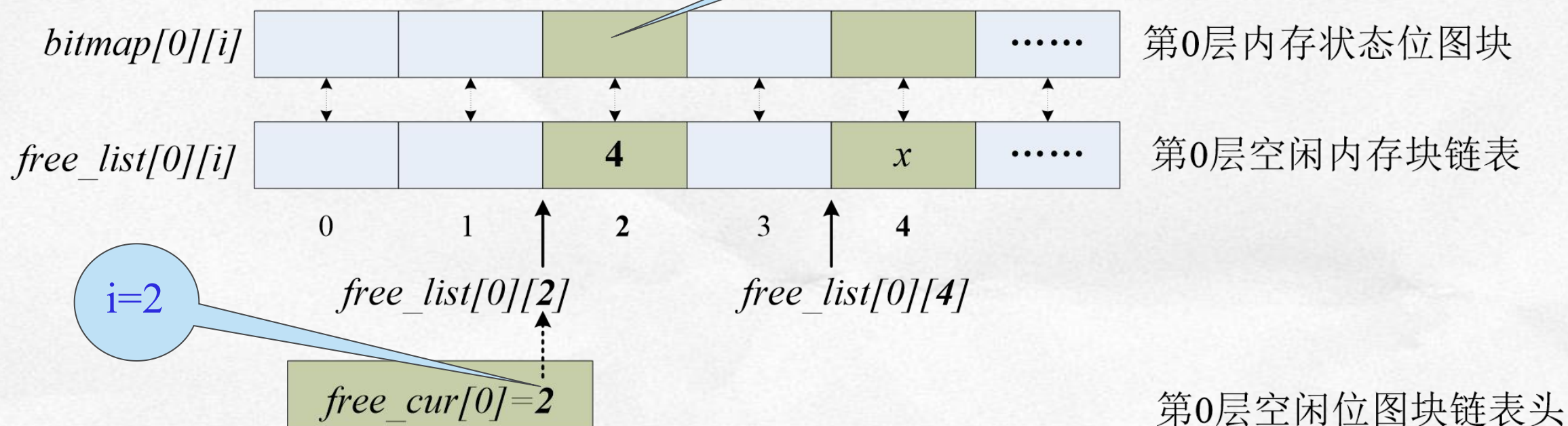
aCoral的第一级管理算法：三种结构的作用 (2)

- 空闲内存块链表头 $free_cur$ ：某层空闲内存块链表头

➤ 指出了本层第一个空闲位图块 (值1)

- 空闲内存块链表 $free_list$ ：某层空闲内存块链表

➤ 指向该层下一个空闲位图块的标号



aCoral的第一级管理算法：最小内存控制块

```
typedef struct{  
    acoral_8 level;          // 对应基本内存块  
}acoral_block_t;
```

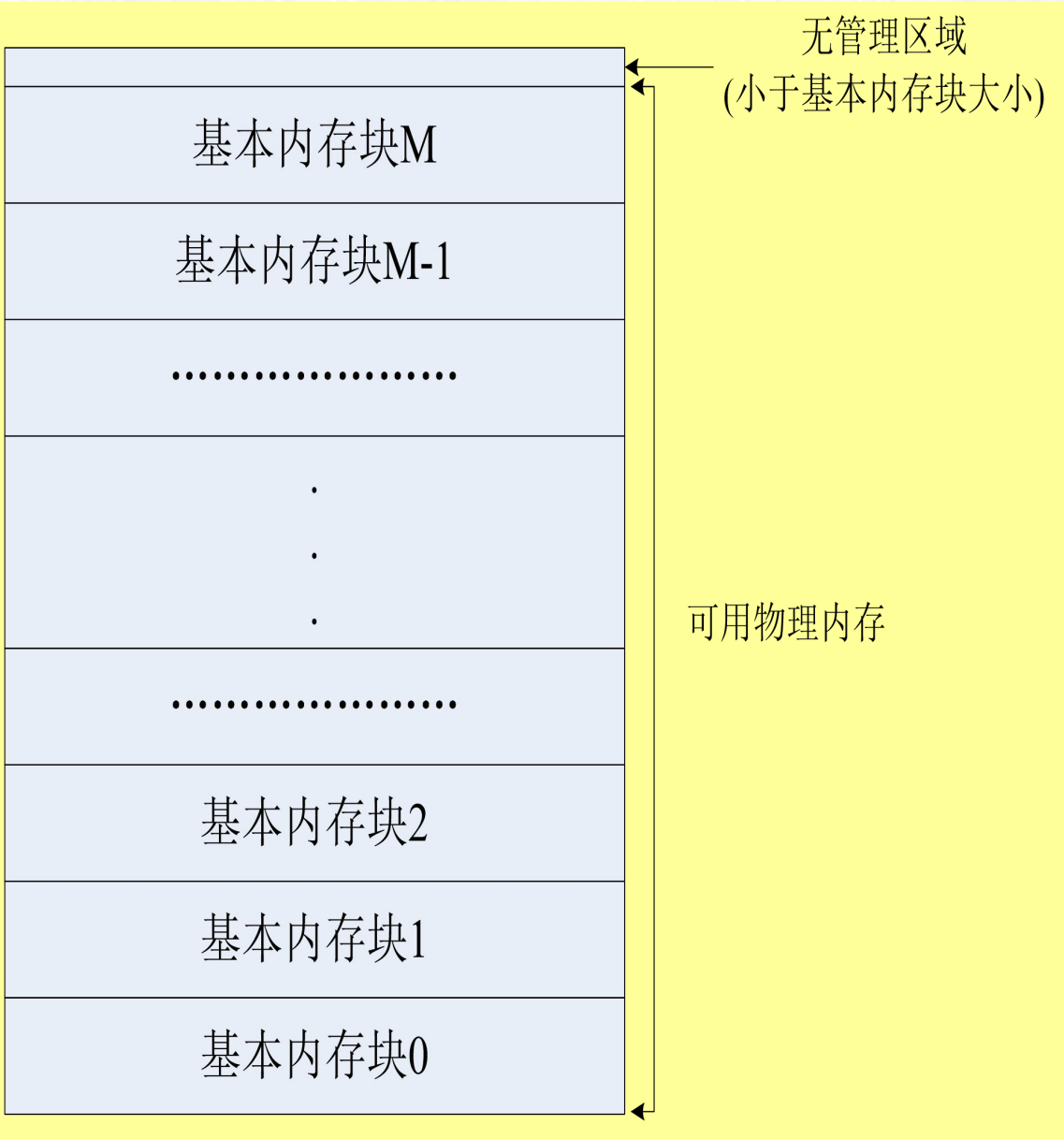
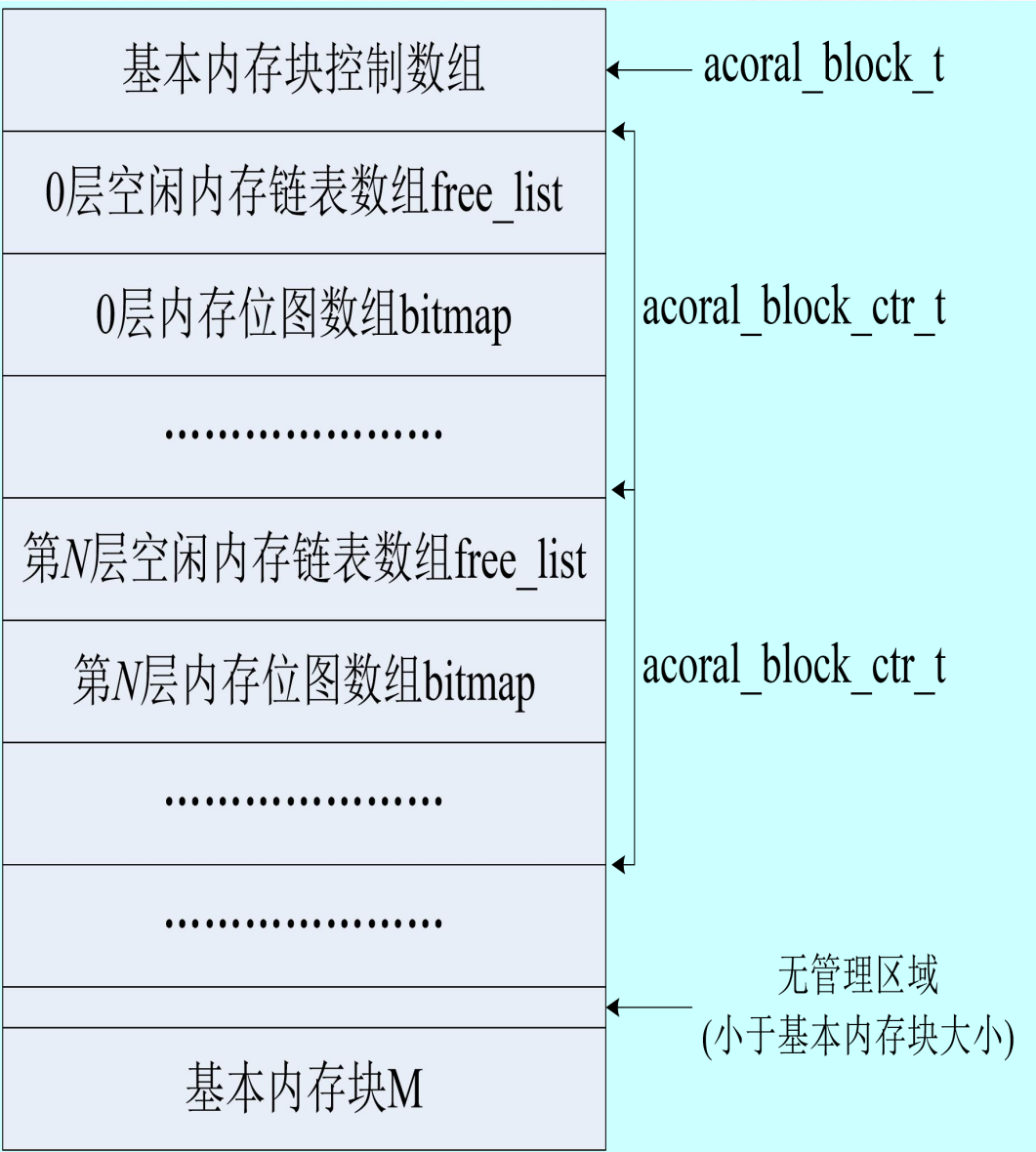
- 每个基本内存块有一个控制结构
- **level = -1** : 该基本内存块尚未分配出去
- **level > 0** : 属于第level层
- **bitmap[m][n]的某一位为1** : 1~m-1层, 偶数块使用, **level > 0**;

奇数的基本内存块只能由
0层分配出去

aCoral的第一级管理算法：存储初始化流程



aCoral的第一级管理算法：初始化后的存储



aCoral的第一级管理算法：初始化

```
acoral_err buddy_init(acoral_u32 start_adr,acoral_u32 end_adr){  
    .....  
    start_adr+=3;                //堆heap起始地址： 四字节对齐  
    start_adr&=~(4-1);           //如3调整到4  
    end_adr&=~(4-1);             // 堆结束： 如7调整为4  
    resize_size=BLOCK_SIZE;     // 初始层块大小  
    if(start_adr>end_adr||end_adr-start_adr<BLOCK_SIZE){ //无法分配  
        state=MEM_NO_ALLOC;  
        return -1;  
    }  
    while(1){                    //获得最大层数  
        if(end_adr<=start_adr+resize_size) break;  
        resize_size=resize_size<<1; //上层块大小  
        num=num<<1;                //基本内存块数： num初始为1  
        adjust_level++;            //层数  
    }  
}
```

aCoral的第一级管理算法：初始化（2）

//为最小内存控制块（0层）分配空间

```
acoral_mem_pages=(acoral_page_t *)end_adr-num;
```

```
save_adr=(acoral_u32)acoral_mem_pages;
```

```
level=adjust_level;
```

```
if(adjust_level>LEVEL) level=LEVEL; //最大
```

```
num=num/32;
```

acoral_block_ctr_t起始位置（给0层用）

num个最小内存块，每个占1结构空间（acoral_block_t, 1字节）

aCoral的第一级管理算法· 初始化 (3)

```
for(i=0;i<level-1;i++){
```

```
    num=num>>1;
```

```
    if(num==0) num=1;
```

```
    save_adr-=num*4;
```

```
    save_adr&=~(4-1);
```

```
    acoral_mem_ctrl->bitmap[i]=(a
```

```
    acoral_mem_ctrl->num[i]=num;
```

```
    save_adr-=num*4;
```

```
    save_adr&=~(4-1);
```

```
    acoral_mem_ctrl->free_list[i]=(acoral_u8 *)save_adr;
```

```
    for(k=0;k<num;k++){
```

```
        acoral_mem_ctrl->bitmap[i][k]=0;
```

```
        acoral_mem_ctrl->free_list[i][k]=-1;
```

```
    }
```

```
    acoral_mem_ctrl->free_cur[i]=-1; //初始化空闲位图块链表头
```

```
}
```

num个数除2 —— 当前层

的控制块数：正好满足

32位管控64个内存块

内存块的空间

// 内存状态位图块bitmap的空间

例如：0层num个bitmap块；

每块32位=4*8；总数

num*4

//空闲位图块的空间

//初始化

-1：不可用

aCoral的第一级管理算法：初始化（4）

//为顶层（第m层的三个结构分配存储空间并初始化）

```
if(num==0) num=1;
```

```
save_adr-=num*4;
```

```
save_adr&=~(4-1);
```

```
acoral_mem_ctrl->bitmap[i]=(acoral_u8 **)save_adr;
```

```
//i=level-1
```

```
acoral_mem_ctrl->num[i]=num;
```

```
save_adr-=num*4;
```

```
save_adr&=~(4-1);
```

```
acoral_mem_ctrl->free_list[i]=(acoral_u8 *)save_adr;
```

```
for(k=0;k<num;k++){
```

```
    acoral_mem_ctrl->bitmap[i][k]=0;
```

```
    acoral_mem_ctrl->free_list[i][k]=-1;
```

```
}
```

```
acoral_mem_ctrl->free_cur[i]=-1;
```

aCoral的第一级管理算法：初始化 (5)

```
//确定管理实际可用内存所需要的逻辑层数、以及这些内存开始、终止地址、  
//基本内存块的个数等数据：预估层数可能未扣除内存控制块所占空间  
if(save_adr<=(start_adr+(resize_size>>1)))  adjust_level--;  
if(adjust_level>LEVEL)  level=LEVEL;           //确定层数  
acoral_mem_ctrl->level=level;                   //逻辑层数  
acoral_mem_ctrl->start_adr=start_adr;           //内存开始  
num=(save_adr-start_adr)>>BLOCK_SHIFT;  
acoral_mem_ctrl->end_adr=start_adr+(num<<BLOCK_SHIFT); //内存结束  
acoral_mem_ctrl->block_num=num;                 //基本内存块个数  
acoral_mem_ctrl->free_num=num;                  //空闲块数  
  
i=0;  
acoral_u32 max_num,o_num;  
max_num=1<<level-1;  
o_num=0;  
if(num>0)           //设置最高层空闲位图块链表头的值
```

aCoral的第一级管理算法：初始化（6）

```
if(num>0)    //设置最高层空闲位图块链表头的值
    acoral_mem_ctrl->free_cur[level-1]=0;    //指向该层的0号空闲内存块
else
    acoral_mem_ctrl->free_cur[level-1]=-1;
//设置最高层bitmap、空闲内存块链表数组：最高层max_num*32为一组
while(num>=max_num*32){
    acoral_mem_ctrl->bitmap[level-1][i]= -1;    //每位置1，均空闲；管理32位
    acoral_mem_ctrl->free_list[level-1][i]=i+1;    //指向下一位：空闲
    num-=max_num*32;
    o_num+=max_num*32;
    i++;
}
```

aCoral的第一级管理算法：初始化 (7)

// level-1层的内存块数不足max_num*32时的处理：32是bitmap标准位数

if(num==0)

 acoral_mem_ctrl->free_list[level-1][i-1]=-1; // i值来自上页循环

while(num>=max_num){

 index=o_num>>level-1;

 acoral_set_bit(index,acoral_mem_ctrl->bitmap[level-1]); //按位赋值1

 num-=max_num;

 o_num+=max_num;

}

acoral_mem_ctrl->free_list[level-1][i]=-1;

..... // 略

aCoral的第一级管理算法：分配内存

```
void *buddy_malloc(acoral_u32 size){
    .....
    resize_size=BLOCK_SIZE;
    if(state==MEM_NO_ALLOC)    return NULL;           //不分配状态
    while(resize_size<size){    //本层块大小不满足申请内存：向上
        num=num<<1;
        level++;
        resize_size=resize_size<<1;
    }
    if(num>acoral_mem_ctrl->free_num) return NULL;    //剩余内存不足
    //申请内存块大小超过顶层内存块大小
    if(level>=acoral_mem_ctrl->level) return NULL;
    return r_malloc(level);    //实际的分配函数
}
```

aCoral的第一级管理算法：实际分配

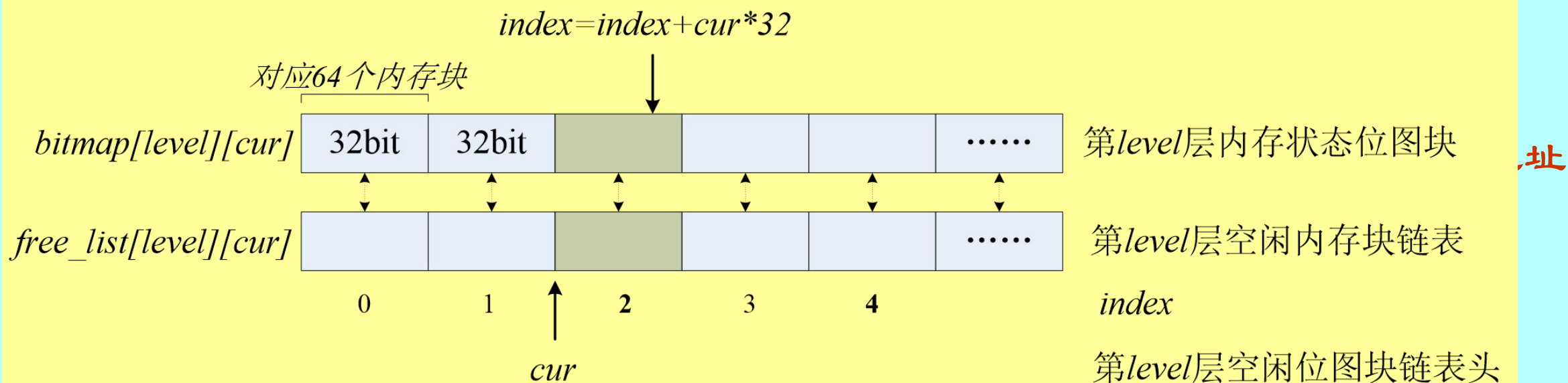
```
void *r_malloc(acoral_u8 level){
    .....
    HAL_ENTER_CRITICAL();
    acoral_spin_lock(&acoral_mem_ctrl->lock);
    acoral_mem_ctrl->free_num-=1<<level;           //该层的空闲内存块数
    acoral_32 cur=acoral_mem_ctrl->free_cur[level]; //free_cur指向的空闲位
    if(cur<0){                                       //该层没有空闲块，向上申请
        num=recus_malloc(level+1); //空闲内存块对应的第一个基本内存块号
        if(num<0) return NULL; //上层没有
        index=num>>level+1;
        cur=index/32;                               //上层空闲位
        acoral_set_bit(index,acoral_mem_ctrl->bitmap[level]); //空闲内存对应的位图位置
        acoral_mem_ctrl->free_list[level][cur]=-1;
        acoral_mem_ctrl->free_cur[level]=cur;        //空闲链表表头指向
    }
}
```

aCoral的第一级管理算法：实际分配 (2)

`acoral_mem_ctrl->free_cur[level]=cur;` // 链表头指向

`if(num%2==0)` //基本内存块开始的内存块是从level层分配

`acoral mem pages[PAGE INDEX(num)].level=level;`



`index=acoral_ffs(acoral_mem_ctrl->bitmap[level][cur]);`

`index=index+cur*32;` //index为第一个非零位对应的内存块序号

aCoral的第一级管理算法：实际分配 (3)

```
//cur为该层空闲内存块链表头；(cur*32)为对应的内存位图块之包含了多少个“1”
acoral_clear_bit(index,acoral_mem_ctrl->bitmap[level]);           //内存块已用：清标记“1”
if(acoral_mem_ctrl->bitmap[level][cur]==0){
    //修改空闲链表：表头指向下一空闲块
    acoral_mem_ctrl->free_cur[level]=acoral_mem_ctrl->free_list[level][cur];
}
acoral_spin_unlock(&acoral_mem_ctrl->lock);
HAL_EXIT_CRITICAL();
if(level==acoral_mem_ctrl->level-1){    //最高层处理
    num=index<<level;                    // num为基本内存块序号
    if(num+(1<<level-1)>acoral_mem_ctrl->block_num)
        return NULL;
}
else{
```

aCoral的第一级管理算法：实际分配 (4)

```
else{
    // 0~(level-2)层， bitmap的某一位代表了相邻的两个内存块
    num=index<<level+1;           // num为基本内存块序号
    if(acoral_mem_pages[PAGE_INDEX(num)].level>=0)
        num += (1<<level);
}
if(num%2==0)                      //基本内存块开始的内存块是从level层分配
    acoral_mem_pages[PAGE_INDEX(num)].level=level;

// 分配的内存块首地址
return acoral_mem_ctrl->start_adr+(num<<BLOCK_SHIFT);
}
```

aCoral的第一级管理算法：空闲的第一个内存块号

```
static acoral_32 recurs_malloc(level){
    .....
    if(level>=acoral_mem_ctrl->level) return -1;
    cur=acoral_mem_ctrl->free_cur[level];           //空闲表头
    if(cur<0){                                       // 本层无空闲块，向上申请
        num=recurs_malloc(level+1);                // 递归处理
        if(num<0) return -1;                        //无空闲块
        index=num>>level+1;
        cur=index/32;
        //该层的空闲内存块对应了该层bitmap的哪一位
        acoral_set_bit(index,acoral_mem_ctrl->bitmap[level]);
        acoral_mem_ctrl->free_list[level][cur]=-1;
        acoral_mem_ctrl->free_cur[level]=cur;
        return num;
    }
}
```

aCoral的第一级管理算法：空闲的第一个内存块号

```
index=acoral_ffs(acoral_mem_ctrl->bitmap[level][cur]);  
index=cur*32+index; //第一个空闲块序号  
acoral_clear_bit(index,acoral_mem_ctrl->bitmap[level]); //清该空闲块标记  
if(acoral_mem_ctrl->bitmap[level][cur]==0) //调整空闲表头  
    acoral_mem_ctrl->free_cur[level]=acoral_mem_ctrl->free_list[level][cur];  
num=index<<level+1;  
if(level==acoral_mem_ctrl->level-1){ //最高层level的处理  
    if((num>>1)+(1<<level)>acoral_mem_ctrl->block_num) return -1;  
    return num>>1;  
}  
if(acoral_mem_blocks[BLOCK_INDEX(num)].level>=0) //确定空闲的伙伴号  
    return num+(1<<level);  
else  
    return num;  
}
```

aCoral的第一级管理算法：内存回收

- **基本思想：**

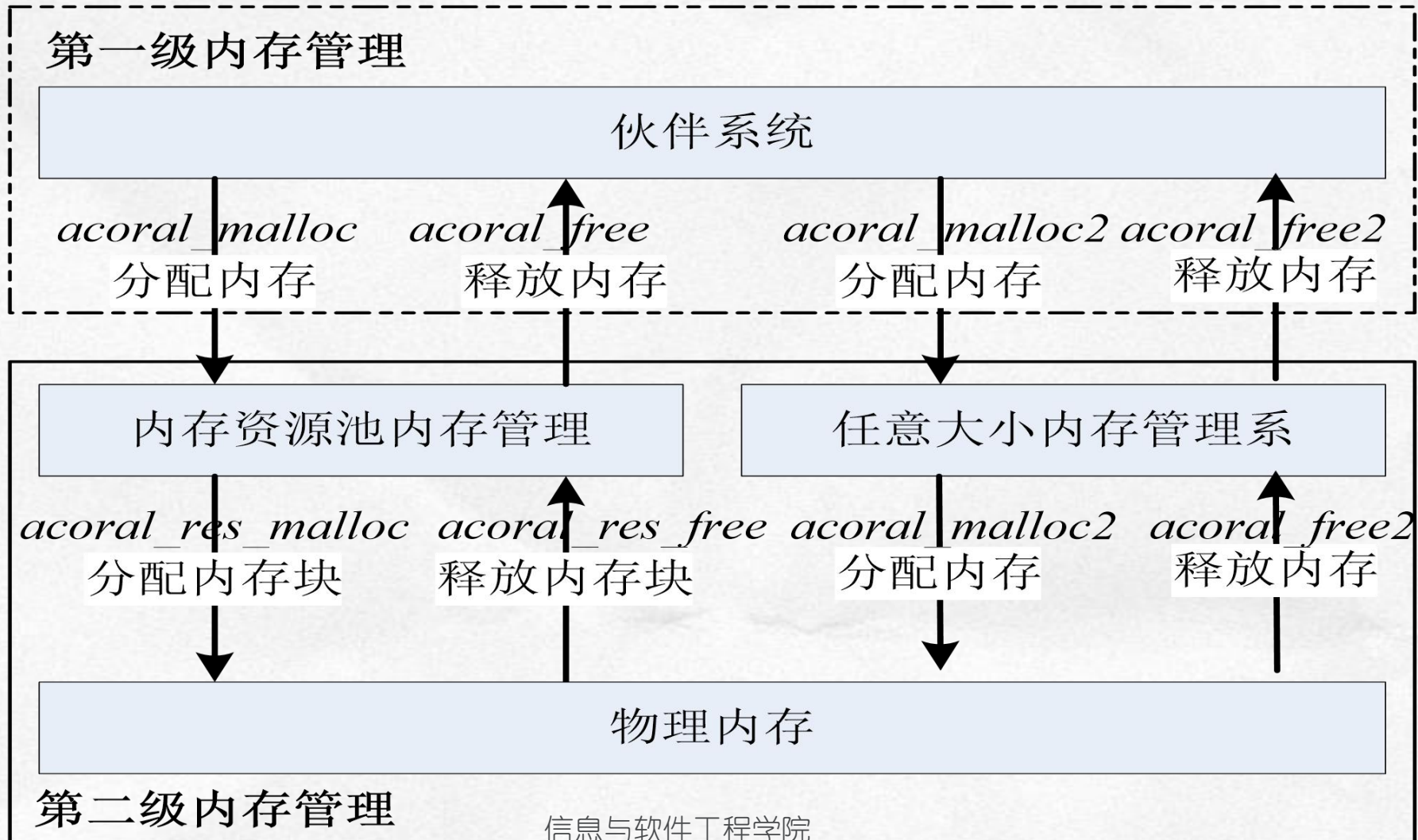
- **回收准备**：状态；回收地址；... → **支持回收操作**
- **获取基本内存块序号**：依回收内存块的起始地址，
- **获取该内存块所属层数**：
- **进入临界区**：
- **合法性检查**：地址、伙伴是否回收、已经回收、...
- **顶层处理**：回收内存，合并伙伴；**不向上回收**
- **非顶层**：回收内存；判断
- **若伙伴未用**：**向上递归合并伙伴**；否则不向上处理

aCoral的第二级处理

- **原则：**固定块和可变大小可选
- **固定大小内存管理：**内存资源池储存管理
 - **针对频繁使用的内存：**如TCB、ECB（事件控制块）、...
 - **方法：**事先对常用结构等分配一些固定内存块，形成资源池，以指针形式分配或回收
- **任意大小内存管理：**需要多少分配多少（**存在！**）
 - **“一分为二”：**将一块分为两块，一块分配给用户使用，剩下一块留作后续使用，同时改变大小标志
 - **存在问题：**不断调用acoral_malloc2()会产生大量内存碎片

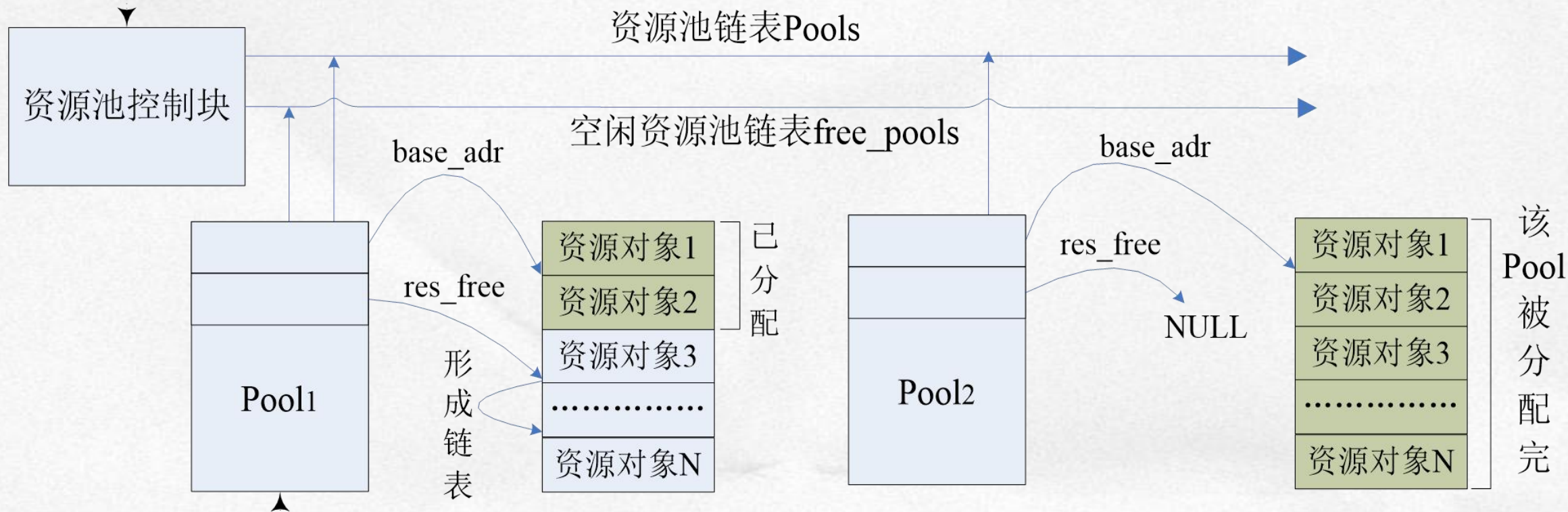
aCoral的内存管理模式

- 原则：两级管理模式



aCoral的第二级处理：内存资源池管理

负责管理一类资源池
(可多个池)



供某一类资源使用(可
分配N个资源对象)

aCoral的第二级处理：资源池控制块结构

```
typedef struct {  
    acoral_u32    type;           //资源类型：TCB、ECB、...  
    acoral_u32    size;          //资源大小：sizeof(TCB)、...  
    acoral_u32    num_per_pool;  //池中对象个数：伙伴关系算法  
                                //可分配大小，满足用户要求  
    acoral_u32    num;           //已经分配的资源池数  
    acoral_u32    max_pools;     //最大可分配的资源池数  
    acoral_list_t *free_pools,*pools_list[2]; //资源池、空闲池链表  
    acoral_res_api_t *api;       //资源操作接口  
#ifdef CFG_CMP  
    acoral_spinlock_t lock;      //自旋锁  
#endif  
    acoral_u8 *name;            //资源名称  
}acoral_pool_ctrl_t;
```

aCoral的第二级处理：资源池结构

```
typedef struct {  
    void *base_adr;           //所管理资源的基地址；或指向下一个资源池（空闲时）  
    void *res_free;           //该池所管理空闲资源对象的链表头  
    acoral_id id;             //该池id  
    acoral_u32 size;          //该池管理的资源大小，=pool_ctrl的size  
    acoral_u32 num;           //管理的资源的个数，=pool_ctrl的num_per_pool  
    acoral_u32 position;      //暂未用  
    acoral_u32 free_num;      //空闲资源个数  
    acoral_pool_ctrl_t *ctrl; //该池对应的资源池控制块  
    acoral_list_t ctrl_list;  //资源池链表节点  
    acoral_list_t free_list;  //空闲资源池链表节点  
#ifdef CFG_CMP  
    acoral_spinlock_t lock;    //自旋锁  
#endif  
}acoral_pool_t;
```

aCoral的第二级处理：资源池管理

- **基本操作**：创建、初始化、获取、释放、....
- **创建内存池**：

```
acoral_err acoral_create_pool(acoral_pool_ctrl_t *pool_ctrl){
    acoral_pool_t *pool;
    if(pool_ctrl->num>=pool_ctrl->max_pools)    return ACORAL_RES_MAX_POOL;
    pool=acoral_get_free_pool();                // 获取一个空闲资源池结构体
    if(pool==NULL) return ACORAL_RES_NO_POOL;
    pool->id=pool_ctrl->type<<ACORAL_RES_TYPE_BIT|pool->id;    //初始化参数
    pool->size=pool_ctrl->size;
    pool->num=pool_ctrl->num_per_pool;
    pool->base_adr=(void *)acoral_malloc(pool->size*pool->num); //分配所用内存
    if(pool->base_adr==NULL)    return ACORAL_RES_NO_MEM;
```

aCoral的第二级处理：资源池管理：创建（2）

// 初始化资源内存池空闲资源链表、空闲资源池数量及其控制块

```
pool->res_free=pool->base_adr;
```

```
pool->free_num=pool->num;
```

```
pool->ctrl=pool_ctrl;
```

```
acoral_pool_res_init(pool);           // 初始化内存池管理的资源对象
```

// 加入资源池链表和空闲链表

```
acoral_list_add2_tail(&pool->ctrl_list,pool_ctrl->pools);
```

```
acoral_list_add2_tail(&pool->free_list,pool_ctrl->free_pools);
```

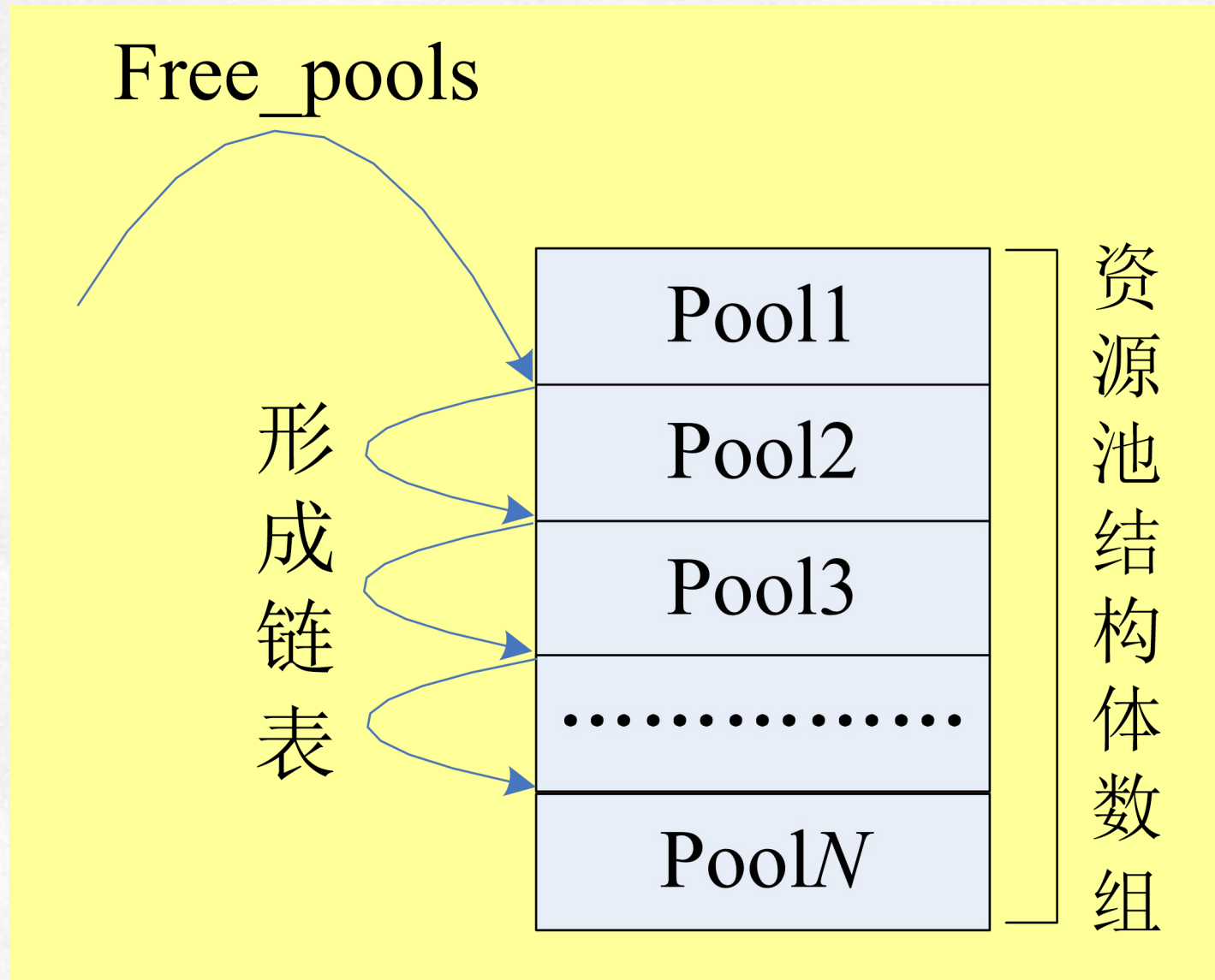
```
pool_ctrl->num++;                     //此控制块管理的池个数
```

```
return 0;
```

```
}
```

aCoral的第二级处理：资源池管理：内存池初始化

- 资源池结构：一个数组
- 快速处理要求：链表



aCoral的第二级处理：资源池管理：内存池初始化 (2)

```
void acoral_pool_res_init(acoral_pool_t * pool){ // 构造成链表
    .....
    blks=pool->num; //资源池Pool的个数
    res=(acoral_res_t *)pool->base_adr; // 该池的基地址
    pblk=(acoral_u8 *)pool->base_adr + pool->size; // 该池空间
    for (i = 0; i < (blks - 1); i++) { // 形成链表
        res->id=i<<ACORAL_RES_INDEX_INIT_BIT; //确定每个池的id
        res->next_id=i+1; //链接下一个id：下一个空闲链表指针
        res=(acoral_res_t *)pblk;
        pblk+=pool->size;
    }
    res->id=blks-1<<ACORAL_RES_INDEX_INIT_BIT;
    res->next_id=0; //最后一个资源池结构为NULL，表示链表尾
}
```

aCoral的第二级处理：资源池管理：分配

```
acoral_res_t *acoral_get_res(acoral_pool_ctrl_t *pool_ctrl){  
    .....  
    HAL_ENTER_CRITICAL();  
    acoral_spin_lock(&pool_ctrl->lock);  
    first=pool_ctrl->free_pools->next;           //获取一个空闲池  
    if(first== pool_ctrl->free_pools){           //无空闲池  
        if(acoral_create_pool(pool_ctrl)){       //创建新空闲内存池  
            acoral_spin_unlock(&pool_ctrl->lock);  
            HAL_EXIT_CRITICAL();  
            return NULL;  
        }  
        else  
            first=pool_ctrl->free_pools->next;    // 空闲资源池节点  
    }  
    pool=list_entry(first,acoral_pool_t,free_list); //获取first的结构地址
```

双向链表：队列

aCoral的第二级处理：资源池管理：分配（2）

```
pool=list_entry(first,acoral_pool_t,free_list);    //获取first的结构地址
res=(acoral_res_t *)pool->res_free;                //空闲资源池结构指针
// 获取资源对象
pool->res_free=(void *)((acoral_u8 *)pool->base_adr+res->next_id*pool->size);
//获取该资源id
res->id=(res->id>>(ACORAL_RES_INDEX_INIT_BIT-ACORAL_RES_INDEX_BIT))
        &ACORAL_RES_INDEX_MASK|pool->id;
pool->free_num--;    //空闲资源对象减一
if(!pool->free_num)    //为0则退出空闲资源池链表
    acoral_list_del(&pool->free_list);
acoral_spin_unlock(&pool_ctrl->lock);
HAL_EXIT_CRITICAL();
return res;
```

```
}
```

aCoral的第二级处理：资源池管理：释放



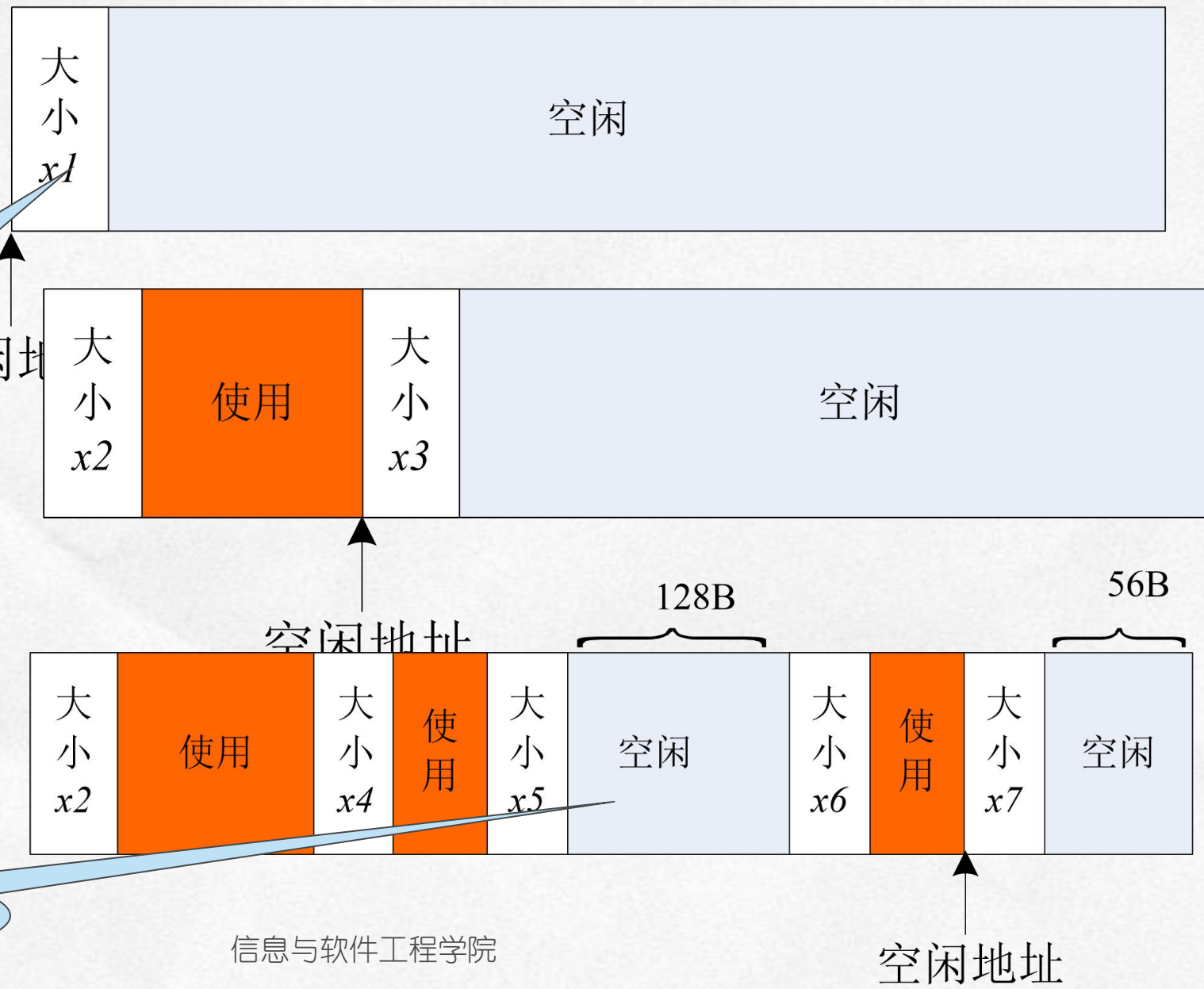
aCoral的第二级处理：任意大小内存管理

- 分配思想：找到第一个可分配块，“一分为二”

- 初始：全部空闲块
控制块：大小为x1

- 多次分配、以及回收：中间形成若干空闲块

- 回收思想：回收、回收后



思考题

- 1、请简述伙伴关系管理算法的基本思想
- 2、aCoral的第二级内存管理机制采用什么方式？
资源池控制块、资源池、资源对象之间的关系是什么？资源池控制块如何定义？当aCoral要为用户分配一个线程控制块时，如何通过资源池控制块找到空闲的内存单元？



谢谢！